Лабораторная работа №3. Docker Images. Образы и реестры

Тема: Получение, исследование, тегирование и публикация Docker-образов

Цель: Научиться работать с Docker Images и Docker Hub: скачивать образы, анализировать слои и метаданные, управлять тегами и публиковать образ.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №3. Docker Images. Образы и реестры. Продолжительность: 4 академических часа (180 минут). Работа выполняется на Debian 13 с установленным Docker Engine. Содержание соответствует лекции №3.

2. Цель и планируемые результаты

Цель: научиться получать Docker-образы, изучать их свойства и историю, назначать теги, удалять локальные ссылки и публиковать образы в Docker Hub.

1. Различать Docker Image, Docker Container, registry, repository, tag, digest.
2. Использовать pull, image ls, inspect, history, tag, rmi.
3. Понимать, что несколько тегов могут ссылаться на один IMAGE ID.
4. Подготовить и опубликовать образ в Docker Hub при наличии учётной записи и доступа.
5. Диагностировать типовые ошибки скачивания и публикации.

3. Оборудование и исходная среда

6. Виртуальная машина Debian 13 с Docker Engine, установленным по предыдущей лабораторной.
7. Терминал и доступ в интернет к Docker Hub.
8. Учётная запись Docker Hub для задания с публикацией.
9. Достаточно свободного места для скачивания образов nginx:stable и alpine:3.21.

На занятии выполняйте Docker-команды с sudo. Не добавляйте учебного пользователя в группу docker ради этой лабораторной: членство в ней предоставляет широкие системные привилегии. Команды выполняются на учебной ВМ, не на продуктивном сервере.

4. Исходная схема

Логическая схема работы

```
Docker Hub (registry)
       | docker pull
       v
Debian 13 / Docker Engine
       |-- nginx:stable
       |-- alpine:3.21
       |-- classroom/alpine:lab1 (дополнительный тег)
       |
       +-- docker push --> личный репозиторий Docker Hub
```

На первом этапе образы остаются на локальной виртуальной машине. В последнем задании выбранному образу назначается имя личного репозитория Docker Hub и выполняется публикация.

5. Подготовка среды

Шаг 1. Проверка Docker

Откройте терминал Debian 13 и убедитесь, что сервер Docker доступен.

```
Проверка клиента и сервера
sudo docker version
```

Пояснение: Команда показывает версии Docker Client и Docker Server.

Ожидаемый результат: В выводе присутствуют обе секции Client и Server.

Критерий перехода: Нет ошибки соединения с Docker Daemon.

Шаг 2. Проверка службы

При необходимости проверьте состояние службы в systemd.

```
Проверка dockerd
sudo systemctl is-active docker
```

Пояснение: is-active сообщает состояние службы.

Ожидаемый результат: Обычно выводится active.

Критерий перехода: Служба активна; при неактивной службе обратитесь к преподавателю и проверьте предыдущую лабораторную.

6. Основная часть

Этап 1. Получение готовых образов

Шаг 3. Скачивание Nginx

Получите готовый образ без запуска контейнера.

```
Скачивание nginx:stable
sudo docker pull nginx:stable
```

Пояснение: pull загружает образ и недостающие слои из Docker Hub. После имени указан тег stable.

Ожидаемый результат: Статус скачивания завершён, приведена ссылка nginx:stable.

Критерий перехода: Нет сообщения об ошибке; образ появился локально.

Шаг 4. Скачивание Alpine

Для сравнения получите второй образ небольшого Linux-дистрибутива.

```
Скачивание alpine:3.21
sudo docker pull alpine:3.21
```

Пояснение: alpine — имя репозитория; 3.21 — тег версии.

Ожидаемый результат: Видно завершение скачивания образа либо сообщение о наличии актуального локального образа.

Критерий перехода: Команда завершилась успешно.

Шаг 5. Просмотр образов

Сравните полученные образы по размеру и идентификаторам.

```
Список локальных образов
sudo docker image ls
```

Пояснение: image ls показывает REPOSITORY, TAG, IMAGE ID и SIZE.

Ожидаемый результат: В списке есть nginx:stable и alpine:3.21.

Критерий перехода: Оба образа найдены, значения записаны в отчёт.

Таблица наблюдений по образам

Образ	Тег	IMAGE ID	Размер
nginx	stable	Заполнить по выводу	Заполнить по выводу
alpine	3.21	Заполнить по выводу	Заполнить по выводу

Размер и IMAGE ID зависят от даты загрузки, версии и архитектуры. Не копируйте чужие значения: фиксируйте фактический вывод своей системы.

Этап 2. Исследование метаданных и слоёв

Шаг 6. Метаданные образа

Изучите структуру JSON, возвращаемого Docker.

```
Метаданные nginx
sudo docker image inspect nginx:stable
```

Пояснение: inspect показывает конфигурацию и метаданные локального образа.

Ожидаемый результат: В JSON видны Id, RepoTags, Architecture, Os и Config.

Критерий перехода: Найдены и выписаны значения Architecture, Os, RepoTags.

Шаг 7. Краткий вывод идентификатора

Получите только Image ID.

```
Форматированный вывод
sudo docker image inspect nginx:stable --format "{{.Id}}"
```

Пояснение: --format задаёт формат вывода; двойные фигурные скобки относятся к шаблону Docker CLI.

Ожидаемый результат: Одна строка с идентификатором sha256:...

Критерий перехода: Значение сопоставлено с IMAGE ID из image ls.

Шаг 8. История образа

Исследуйте историю формирования образа.

```
История слоёв
sudo docker history nginx:stable
```

Пояснение: history показывает записи истории, связанные с инструкциями сборки и размерами.

Ожидаемый результат: Таблица с колонками CREATED, CREATED BY, SIZE и другими.

Критерий перехода: В отчёте описаны не менее трёх видимых записей; не считайте каждую строку отдельным файловым слоем.

Шаг 9. Digest образов

Посмотрите наличие digest и сравните с Image ID.

```
Список с digest
sudo docker image ls --digests
```

Пояснение: --digests добавляет колонку DIGEST, если информация доступна.

Ожидаемый результат: Для некоторых строк DIGEST может быть <none>.

Критерий перехода: Студент объяснил различие между локальным Image ID, тегом и digest опубликованного артефакта.

Этап 3. Изменение тегов и удаление ссылки

Шаг 10. Новый тег Alpine

Назначьте локальному образу дополнительное учебное имя.

```
Создать второй тег
sudo docker tag alpine:3.21 classroom/alpine:lab1
```

Пояснение: tag создаёт новую ссылку на тот же образ, не изменяя файловые слои.

Ожидаемый результат: Команда завершилась без вывода.

Критерий перехода: Следующая проверка покажет оба имени.

Шаг 11. Проверка одинакового IMAGE ID

Убедитесь, что новый тег не создал копию.

```
Сравнить список образов
sudo docker image ls
```

Пояснение: Оба имени alpine:3.21 и classroom/alpine:lab1 должны вести к одному Image ID.

Ожидаемый результат: В таблице две строки с одинаковым IMAGE ID.

Критерий перехода: Совпадение зафиксировано скриншотом.

Шаг 12. Удаление дополнительного тега

Удалите только созданную локальную ссылку.

```
Удалить тег lab1
```

```
sudo docker rmi classroom/alpine:lab1
```

Пояснение: rmi удаляет указанную ссылку; исходное имя alpine:3.21 не затрагивается.

Ожидаемый результат: Появится сообщение об удалении тега.

Критерий перехода: Оригинальный образ всё ещё отображается через image ls.

Этап 4. Диагностические эксперименты

Все эксперименты должны быть обратимыми. Не применяйте глобальную очистку образов на общей учебной системе.

Эксперимент А. Несуществующий тег

Преднамеренно запросите тег, которого нет в репозитории.

```
Ошибочный запрос
```

```
sudo docker pull nginx:tag-does-not-exist-lab3
```

Пояснение: Имя тега специально неправильное. Docker обращается к реестру за отсутствующей ссылкой.

Ожидаемый результат: Сообщение об ошибке вроде manifest unknown или not found; точный текст зависит от реестра.

Критерий перехода: Студент объяснил причину, исходные образы сохранены.

Восстановление после эксперимента А

Повторите загрузку с существующим тегом.

```
Корректный pull
```

```
sudo docker pull nginx:stable
```

Пояснение: Исправляется только тег, предварительно введённый неверно.

Ожидаемый результат: Docker подтверждает успешное получение или актуальность локального образа.

Критерий перехода: Исходное состояние восстановлено.

Эксперимент Б. Попытка удалить образ с контейнером

Создайте остановленный контейнер, который зависит от alpine:3.21.

```
Создать остановленный контейнер
```

```
sudo docker create --name lab3-held alpine:3.21 echo test
```

Пояснение: create создаёт контейнер, но не запускает его.

Ожидаемый результат: Выводится идентификатор контейнера.

Критерий перехода: В списке ps -a контейнер lab3-held имеет состояние Created.

Эксперимент Б. Проверка защиты образа

Попробуйте удалить образ, от которого зависит контейнер.

```
Попытка удалить используемый образ
```

```
sudo docker rmi alpine:3.21
```

Пояснение: Если это единственная локальная ссылка на образ, Docker обычно запрещает удаление образа, на который ссылается контейнер.

Ожидаемый результат: Ожидается конфликт удаления; возможные формулировки зависят от версии Docker.

Критерий перехода: Сформулирован вывод: существующий контейнер связан с образом.

Восстановление после эксперимента Б

Удалите только созданный контейнер и затем при желании повторите удаление образа.

```
Удалить тестовый контейнер
```

```
sudo docker rm lab3-held
```

Пояснение: Удаляет ранее созданный, не запущенный контейнер.

Ожидаемый результат: Выводится имя lab3-held.

Критерий перехода: В docker ps -a контейнера нет; alpine:3.21 оставьте для следующих занятий.

Этап 5. Реестр Docker Hub и публикация

Для этого этапа требуется личная учётная запись Docker Hub и разрешение на публикацию в собственный репозиторий. Если в классе регистрация недоступна, преподаватель демонстрирует push на своём тестовом репозитории, а студент документирует команды и выполняет локальную часть. Не используйте чужие учётные данные.

Шаг 13. Подготовка имени

Перед тегированием уточните своё имя пользователя Docker Hub и создайте/выберите репозиторий docker-nginx-lab.

```
Пример имени для публикации
```

```
sudo docker tag nginx:stable student123/docker-nginx-lab:v1
```

Пояснение:
- student123 — УСЛОВНЫЙ пример. Перед выполнением команды замените student123 на собственный Docker ID (имя пользователя в Docker Hub).
- docker-nginx-lab - Название репозитория в Docker Hub
- v1 - Тег (обозначение версии) образа

Ожидаемый результат: После tag появляется локальная ссылка со своим namespace.

Критерий перехода: Репозиторий и имя пользователя указаны корректно.

Шаг 14. Авторизация

Авторизуйтесь поддерживаемым Docker CLI способом.

Вход в Docker Hub

```
sudo docker login
```

Пояснение: При необходимости Docker предложит вход через браузер или запросит учётные данные.

Ожидаемый результат: Сообщение об успешной авторизации.

Критерий перехода: Не включайте пароль, токен или коды входа в отчёт.

Шаг 15. Публикация образа

Загрузите образ в собственный Docker Hub репозиторий.

Отправить образ

```
sudo docker push student123/docker-nginx-lab:v1
```

Пояснение: Перед вводом замените student123 на свой Docker ID и убедитесь в наличии права на push.

Ожидаемый результат: Видно завершение отправки слоёв и digest опубликованного артефакта.

Критерий перехода: Тег v1 доступен на странице репозитория в Docker Hub.

Шаг 16. Проверка публикации

Посмотрите репозиторий на Docker Hub и повторите получение по опубликованному имени.

Проверка pull

```
sudo docker pull student123/docker-nginx-lab:v1
```

Пояснение: Замените student123 на свой Docker ID. pull получает образ по опубликованной ссылке.

Ожидаемый результат: Команда заканчивается успешно и подтверждает наличие ссылки.

Критерий перехода: Образ доступен владельцу аккаунта; репозиторий может быть public или private.

Шаг 17. Выход из аккаунта

После завершения работы выйдите из Docker Hub на общем учебном ПК.

Выйти из Docker Hub

```
sudo docker logout
```

Пояснение: Удаляет сохранённые Docker CLI учётные данные для реестра из текущего контекста.

Ожидаемый результат: Сообщение о выходе либо отсутствии входа.

Критерий перехода: Учётная запись больше не авторизована в использованном CLI-контексте.

7. Итоговое самостоятельное задание

10. Скачать образ redis:7 и записать имя, тег, Image ID и размер.
11. Посмотреть историю образа через docker history.
12. Создать дополнительный локальный тег classroom/redis:practice.
13. Доказать совпадение IMAGE ID исходного и нового имени.
14. Удалить только classroom/redis:practice, оставив redis:7.
15. Сделать вывод о назначении образов, тегов и реестров.

Для самостоятельного задания используйте команды, аналогичные разобранным в работе. Не удаляйте чужие образы и не выполняйте docker image prune -a.

8. Итоговая проверка

```
Проверка списка локальных образов и контейнеров

sudo docker image ls
sudo docker ps -a
```

В конце работы должны сохраниться исходные учебные образы, если преподаватель не дал другое указание. Контейнер lab3-held удалён, временные теги classroom/alpine:lab1 и classroom/redis:practice отсутствуют. В отчёте приложены доказательства каждого этапа.

9. Содержание отчёта

16. Тема, цель, используемая ОС и версия Docker Engine.
17. Таблица размеров, тегов и IMAGE ID образов nginx:stable и alpine:3.21.
18. Скриншоты или текстовый вывод pull, image ls, inspect, history.
19. Подтверждение двух имён с одинаковым IMAGE ID и результата удаления одного тега.
20. Описание эксперимента с несуществующим тегом и восстановлением.
21. Описание эксперимента с контейнером lab3-held и восстановлением.
22. Подтверждение публикации в Docker Hub (если аккаунт доступен), без секретов.
23. Результаты самостоятельного задания с redis:7.
24. Краткий вывод: различия образа, контейнера, реестра, тега и digest.

10. Контрольные вопросы

25. 1. Чем образ Docker отличается от контейнера?
26. 2. Что хранится в Docker Registry?
27. 3. Какую задачу решает команда docker pull?
28. 4. Как связаны слои образа и экономия дискового пространства?
29. 5. Почему docker history не всегда совпадает со списком файловых слоёв?
30. 6. Что показывает команда docker image inspect?
31. 7. Для чего используются Image ID и digest?
32. 8. Почему тег latest нельзя считать фиксированной версией?
33. 9. Чем docker tag отличается от копирования образа?
34. 10. Почему docker rmi может вернуть ошибку конфликта?
35. 11. Что необходимо для успешного docker push?
36. 12. Почему нельзя помещать пароль Docker Hub в отчёт?

37. 13. Чем отличаются публичный и приватный репозитории?

11. Источники

38. Docker Docs — docker image: https://docs.docker.com/reference/cli/docker/image/
39. Docker Docs — docker image pull: https://docs.docker.com/reference/cli/docker/image/pull/
40. Docker Docs — docker image tag: https://docs.docker.com/reference/cli/docker/image/tag/
41. Docker Docs — Push images to a repository: https://docs.docker.com/docker-hub/repos/manage/hub-images/push/